PROGRAMACIÓN I · UNIDAD 07

Unidad 7. Clases y objetos: representar problemas del mundo real

Programación I — 2026
1.º Bachillerato Tecnológico en Informática · DGETP / UTU

Presentación: aprender a modelar, no solamente a escribir clases

Hasta ahora resolvimos problemas con variables, condiciones, ciclos, arreglos y métodos. Esas herramientas siguen siendo necesarias. La novedad de esta unidad consiste en cambiar la pregunta inicial: además de preguntarnos **qué instrucciones debemos ejecutar**, nos preguntaremos **qué entidades intervienen en el problema, qué información necesitamos conservar de cada una y qué responsabilidades les corresponden**.

Imaginemos una tienda con productos que tienen código, nombre, precio y stock. Podemos usar varias variables o arreglos paralelos, pero ¿cómo mantenemos junta la información de cada producto? ¿Y dónde conviene situar las reglas para agregar o retirar unidades?

Aprenderemos a construir modelos útiles de esa realidad, convertirlos en clases Java y crear objetos con estados independientes. **Una clase no será la traducción automática de un sustantivo**, y una buena solución no será la que contenga más clases, getters o setters.

Al finalizar deberás poder: identificar entidades y justificar su selección; distinguir clases, objetos y referencias; implementar atributos, constructores y métodos de instancia; explicar cómo cambia el estado; introducir `private` y operaciones validadas cuando el problema lo necesite; probar la solución y defender las decisiones de modelado.

Método de trabajo: problema → identificar entidades → seleccionar atributos y responsabilidades → pseudocódigo → Java → predicción → pruebas → explicación. Trabajaremos primero con una clase muy sencilla y luego la evolucionaremos sin adelantar innecesariamente contenidos avanzados.

> **Convención para los ejemplos.** Cuando aparece **«Programa completo»**, los archivos indicados pueden copiarse en la misma carpeta de VS Code y ejecutarse con Java. Cuando aparece **«Fragmento para analizar»**, ese código depende del contexto que se explica y no debe copiarse aisladamente. Algunos fragmentos tienen errores intencionales claramente señalados.

Cómo utilizar este capítulo de forma autónoma. Cuando encuentres una pregunta de predicción, escribe o piensa tu respuesta *antes* de revelar la explicación. Si te bloqueas, identifica primero qué datos tienes y qué resultado necesitas; consulta una pista y vuelve a intentarlo. Si ya entendiste el ejemplo, compruébalo con otros datos y explica qué cambiaría. En los ejercicios con soluciones, la página debe mostrar las pistas antes de la resolución completa. Si estudias el Markdown directamente, oculta el párrafo de solución mientras realizas la predicción.

---

1. Una limitación de las soluciones que conocemos

La tienda registra:

Código	Nombre	Precio	Stock
101	Cuaderno	120	15
102	Lápiz	35	40
103	Mochila	950	8

1.1. Primer intento: muchas variables

Java

```
int codigo1 = 101;
String nombre1 = "Cuaderno";
double precio1 = 120;
int stock1 = 15;

int codigo2 = 102;
String nombre2 = "Lápiz";
double precio2 = 35;
int stock2 = 40;
```

Podemos trabajar así, pero **la relación entre los datos depende de nuestros nombres**. Java no reconoce todavía que `codigo1`, `nombre1`, `precio1` y `stock1` describen una única entidad.

1.2. Segundo intento: arreglos paralelos

Java

```
int[] codigos = {101, 102, 103};
String[] nombres = {"Cuaderno", "Lápiz", "Mochila"};
double[] precios = {120, 35, 950};
int[] stocks = {15, 40, 8};
```

Ahora el índice 1 relaciona el código 102, el nombre «Lápiz», el precio 35 y el stock 40. Esto **puede funcionar**. Los arreglos no son «malos»: fueron una buena respuesta cuando necesitábamos conservar muchos valores de un mismo tipo.

Sin embargo, la unidad «producto» está repartida entre cuatro arreglos. Si insertamos información en posiciones diferentes, podemos mezclar datos. Si necesitamos enviar **un producto completo** a otro método, debemos enviar varias piezas y mantener la correspondencia. Si agregamos categoría y descuento, aumentan los arreglos paralelos.

Experimento en papel

Sin cambiar el orden de `nombres`, imagina que alguien intercambia `precios[0]` y `precios[1]`. ¿Qué mostrará un recorrido que imprime nombre y precio con el mismo índice? ¿La computadora detectará automáticamente la asociación incorrecta?

No: el programa puede compilar y ejecutar, aunque la información quede mal asociada.

1.3. La nueva necesidad

Queremos tratar los datos de cada producto como **una entidad reconocible** y asociarle las operaciones que afectan su propio estado: consultar precio, agregar stock o retirar unidades.

No abandonaremos arreglos, ciclos ni métodos. Los reutilizaremos: más adelante podremos tener **un arreglo de objetos**.

Idea central: el criterio para elegir una representación es lo que necesita resolver el programa, no una supuesta superioridad absoluta de una herramienta.

Chequeo: explica por qué un arreglo de diez calificaciones sigue siendo una buena solución y por qué representar productos mediante varios arreglos paralelos puede volverse incómodo.

---

2. Antes de escribir Java: observar, abstraer y decidir

Un análisis orientado a objetos empieza por comprender una situación. No comienza con `public class`.

2.1. Candidatos a objetos

Una biblioteca registra libros, lectores y préstamos. Estos nombres sugieren posibles entidades. Pero no todo sustantivo del enunciado necesita una clase.

En «un estudiante consulta un libro desde una computadora», quizá necesitemos representar `Estudiante` y `Libro`. La computadora podría no formar parte del sistema; «disponibilidad» podría ser un atributo o un resultado, no necesariamente una entidad independiente.

Para decidir, pregunta:

1. ¿Qué necesita hacer el programa?
2. ¿De qué entidades debemos conservar información individual?
3. ¿Qué características son necesarias para esas tareas?
4. ¿Qué comportamientos o reglas corresponden principalmente a cada entidad?
5. ¿Qué elementos podemos dejar fuera de esta versión?

2.2. Abstracción: un mismo objeto real, dos modelos diferentes

Situación A: alquiler de bicicletas. Interesan el identificador, el tipo, la disponibilidad y si está alquilada. Podríamos necesitar operaciones como `alquilar` y `devolver`.

Situación B: mantenimiento de bicicletas. Interesan kilómetros recorridos, estado de frenos y fecha de revisión. Podríamos necesitar `registrarRevision` o `marcarReparacionPendiente`.

No significa que existan «dos bicicletas reales». Significa que **el propósito determina qué aspectos de la realidad incluimos en el modelo**. Ninguno necesita contener todo lo que sabemos sobre una bicicleta.

Actividad guiada: dos estudiantes posibles

Diseña en papel un modelo `Estudiante` para **calificaciones** y otro para **préstamos de biblioteca**. Para cada caso indica tres atributos relevantes y dos que dejarías fuera. Compara: ¿qué información aparece en ambos?, ¿cuál solamente tiene sentido en uno? Justifica tu selección.

2.3. La abstracción continuará durante toda la unidad

Volveremos a identificar entidades **en tres momentos**:

- **Antes de Java:** proponer qué representar sin conocer todavía la sintaxis.
- **Durante la construcción:** revisar si cada atributo y método responde al problema; corregir modelos poco coherentes.
- **Después de aprender las clases:** recibir una situación nueva sin clases predefinidas y diseñar una solución.

Así evitamos que el modelado se reduzca a una actividad introductoria desconectada del código.

2.4. Un error de diseño para discutir

Un supuesto sistema de biblioteca propone:

Texto

```
LIBRO
Atributos: titulo, autor, paginas, sueldoBibliotecario,
            contrasenaAdministrador
Métodos: mostrarLibro, calcularSueldos,
          registrarTodosLosUsuarios, prestarLibro
```

¿Qué responsabilidades parecen pertenecer a un libro? ¿Cuáles necesitarían otra entidad o sistema? ¿Qué dejarías fuera de un programa que solamente permite consultar libros?

No busques simplemente «sustantivos». **Argumenta en función de los requisitos.**

2.5. Recurso interactivo: construir un modelo antes de ver código

> La finalidad determina qué datos representar. Para alquiler de bicicletas pueden importar identificador y disponibilidad; para mantenimiento, kilómetros y estado de frenos. Propón comportamientos y justifica qué dejas fuera.
>
> Predice el resultado y contrástalo con los ejemplos del capítulo.

Antes de continuar: explica en una o dos frases qué significa abstraer. Una respuesta adecuada debería mencionar el **propósito del programa**.

---

3. Clases, objetos, atributos y comportamientos

Imagina los productos:

- Código 101, «Cuaderno», precio 120, stock 15.
- Código 102, «Lápiz», precio 35, stock 40.
- Código 103, «Mochila», precio 950, stock 8.

Todos comparten una **estructura de información**. Cada uno conserva **valores diferentes**.

3.1. Clase: la definición común

Una clase describe un tipo de entidad dentro del programa. Puede declarar atributos, métodos y constructores. Para este problema, una clase `Producto` podría tener:

Texto

```
PRODUCTO
Datos: codigo, nombre, precio, stock
Posibles operaciones: mostrarInformacion, agregarStock, retirarStock
```

Esta descripción no crea tres productos. Solamente expresa qué queremos que los productos representados puedan contener y hacer.

3.2. Objeto: una instancia concreta

Cada producto creado a partir de `Producto` será un **objeto**. El cuaderno y el lápiz son instancias distintas, aunque ambos respondan a la misma clase.

Los **atributos de instancia** conservan el estado de cada objeto. Los **métodos de instancia** describen operaciones que podemos solicitarle a ese objeto.

Pregunta	Respuesta para nuestro ejemplo
¿Qué clase propia?	`Producto`
¿Cuántos productos concretos?	Los que necesite el programa
¿Los objetos comparten la estructura?	Sí
¿Comparten necesariamente los valores?	No
¿Modificar uno cambia automáticamente los demás?	No, si son instancias distintas

3.3. Estado independiente y referencias compartidas

Más adelante crearemos:

Java

```
Producto a = new Producto(101, "Cuaderno", 120, 15);
Producto b = new Producto(102, "Lápiz", 35, 40);
```

Las dos apariciones de `new` crean dos objetos diferentes. Pero:

Java

```
Producto c = a;
```

no crea otro objeto. `c` recibe la misma referencia que `a`. Si una operación modifica el objeto referenciado por `c`, el cambio también se observa desde `a`.

No confundas cantidad de variables de referencia con cantidad de objetos. Este asunto se retomará con código ejecutable en la sección 5.

3.4. Recurso: clase e instancias

> Producto define código, nombre, precio y stock. Dos expresiones new crean objetos independientes; c = a copia una referencia, por lo que c y a observan el mismo objeto.
>
> Predice el resultado y contrástalo con los ejemplos del capítulo.

Pregunta individual: en un programa con la clase `Producto`, dos expresiones `new Producto(...)` y la asignación `Producto c = a`, ¿cuántos objetos se crearon? Justifica sin ejecutar.

---

4. Primera clase Java: del modelo al código

No comenzaremos con encapsulamiento completo, validaciones avanzadas ni excepciones. Primero comprenderemos la estructura básica.

4.1. Proyecto simple en VS Code

Crea una carpeta, por ejemplo `TiendaInicial`, con dos archivos en el mismo directorio:

- `Producto.java`
- `Main.java`

Por ahora no utilizaremos paquetes, Maven ni frameworks. La extensión opcional **Java Extension Pack** puede facilitar la ejecución.

4.2. Archivo Producto.java

Java

```
public class Producto {
    int codigo;
    String nombre;
    double precio;
    int stock;
}
```

Los atributos representan lo que decidimos conservar de un producto. **Todavía no hemos creado ningún producto.**

Estos atributos están escritos **sin modificador de acceso**. En Java eso significa *acceso de paquete* (no `public`). `Main`, ubicado en el mismo paquete, podrá consultarlos y modificarlos directamente en este primer ejercicio. Más adelante los haremos privados.

4.3. Archivo Main.java: primera creación y consulta

Java

```
public class Main {
    public static void main(String[] args) {
        Producto producto = new Producto();
        producto.codigo = 101;
```

```
        producto.nombre = "Cuaderno";
        producto.precio = 120;
        producto.stock = 15;

        System.out.println(producto.nombre);
        System.out.println(producto.stock);
    }
}
```

Salida esperada:

Salida esperada

```
Cuaderno
15
```

4.4. Qué observamos y qué nos falta resolver

El punto `.` permite seleccionar un atributo de ese objeto: `producto.nombre`. En esta primera versión, podemos asignar valores directamente.

Pero podemos olvidarnos de completar un atributo o establecer valores absurdos. **Esta clase es provisional y deliberadamente sencilla**: su finalidad es comprender los objetos, no garantizar todavía todas las reglas de la tienda.

Predicción: si creas otro `Producto` con `new`, ¿tendrá automáticamente el nombre «Cuaderno»?

No: tendrá su propio estado inicial, hasta que le asignemos datos.

---

5. Crear objetos con new y seguir sus estados

La instrucción:

Java

```
Producto producto = new Producto();
```

contiene dos aspectos distintos: `new Producto()` construye un objeto y `producto` es una variable que conserva una referencia a él. No debemos imaginar que la variable contiene físicamente todos los atributos.

5.1. Dos objetos distintos

Programa completo con el `Producto.java` de la sección 4:

Java

```
public class Main {
    public static void main(String[] args) {
        Producto cuaderno = new Producto();
        cuaderno.nombre = "Cuaderno";
        cuaderno.stock = 15;

        Producto lapiz = new Producto();
        lapiz.nombre = "Lápiz";
        lapiz.stock = 40;

        cuaderno.stock = 10;

        System.out.println(cuaderno.stock);
        System.out.println(lapiz.stock);
    }
}
```

La salida será `10` y `40` en líneas distintas. Modificar uno no altera el otro.

5.2. Dos referencias a un mismo objeto

Otro Main.java, con el mismo `Producto.java`:

Java

```
public class Main {
    public static void main(String[] args) {
        Producto a = new Producto();
        a.stock = 15;

        Producto b = a; // No crea otro objeto.
        b.stock = 25;

        System.out.println(a.stock);
        System.out.println(b.stock);
    }
}
```

Ambas líneas muestran `25`. El único `new` creó un solo objeto, compartido por dos referencias.

5.3. Trazado comparativo

Instrucción	Objeto referido por `a`	Objeto referido por `b`
`Producto a = new Producto()`	Objeto 1	—
`a.stock = 15`	Objeto 1, stock 15	—

Instrucción	Objeto referido por `a`	Objeto referido por `b`
`Producto b = a`	Objeto 1	Objeto 1
`b.stock = 25`	Objeto 1, stock 25	Objeto 1, stock 25

Aquí recuperamos el aprendizaje de referencias de las unidades 5 y 6. No necesitamos estudiar todavía mecanismos internos de memoria.

Chequeo de comprensión: explica por qué «dos nombres de variables» no significa necesariamente «dos objetos».

---

6. Constructores: evitar olvidar los datos iniciales

Con el diseño inicial, podemos hacer esto:

Java

```
Producto producto = new Producto();
producto.nombre = "Cuaderno";
producto.precio = 120;
// Olvidamos el código y el stock.
```

La clase existe, pero no hemos completado la información prevista. ¿Podemos solicitar todos los datos al momento de crear el objeto?

6.1. Primer constructor con parámetros

Sustituye Producto.java por esta versión:

Java

```
public class Producto {
    int codigo;
    String nombre;
    double precio;
    int stock;

    public Producto(int codigo, String nombre,
                    double precio, int stock) {
        this.codigo = codigo;
        this.nombre = nombre;
        this.precio = precio;
        this.stock = stock;
    }
}
```

Un constructor tiene el mismo nombre que la clase y **no declara tipo de retorno** (ni siquiera `void`). Los parámetros reciben los datos de creación. `this.codigo` representa el atributo del objeto actual y `codigo` es el parámetro local.

6.2. Crear dos objetos completos

Main.java:

```java
public class Main {
    public static void main(String[] args) {
        Producto cuaderno = new Producto(101, "Cuaderno", 120, 15);
        Producto lapiz = new Producto(102, "Lápiz", 35, 40);

        System.out.println(cuaderno.nombre);
        System.out.println(lapiz.nombre);
    }
}
```

Salida: «Cuaderno» y «Lápiz».

El constructor reemplaza cuatro asignaciones posteriores, pero todavía **no verifica que los valores sean adecuados**. Esa es una limitación que trataremos en las secciones 8–10.

6.3. Una precisión sobre el constructor sin argumentos

Mientras una clase no declara ningún constructor, Java proporciona uno predeterminado sin argumentos. Cuando declaramos únicamente este constructor de cuatro parámetros, `new Producto()` deja de ser válido a menos que también escribamos explícitamente otro constructor sin argumentos.

Ejercicio de predicción: identifica por qué `new Producto(101, "Cuaderno", 120)` no coincide con el constructor que escribimos.

Le falta el cuarto argumento, `stock`.

---

7. Métodos de instancia: qué puede hacer un producto

En la Unidad 6 escribimos métodos `static` en `Main`. Ahora podemos definir **comportamientos asociados a cada objeto**.

7.1. Una responsabilidad cercana a los datos

Un producto conoce sus propios atributos. Un método para mostrarlos puede utilizar directamente ese estado.

Producto.java completo:

Java

```
public class Producto {
    int codigo;
    String nombre;
    double precio;
    int stock;

    public Producto(int codigo, String nombre,
                    double precio, int stock) {
        this.codigo = codigo;
        this.nombre = nombre;
        this.precio = precio;
        this.stock = stock;
    }

    public void mostrarInformacion() {
        System.out.println("Código: " + codigo);
        System.out.println("Nombre: " + nombre);
        System.out.println("Precio: " + precio);
        System.out.println("Stock: " + stock);
    }
}
```

Main.java completo:

Java

```
public class Main {
    public static void main(String[] args) {
        Producto cuaderno = new Producto(101, "Cuaderno", 120, 15);
        Producto lapiz = new Producto(102, "Lápiz", 35, 40);

        cuaderno.mostrarInformacion();
        lapiz.mostrarInformacion();
    }
}
```

La misma definición `mostrarInformacion()` trabaja primero con el estado del cuaderno y luego con el del lápiz. **El objeto que aparece antes del punto determina sobre qué instancia se ejecuta el método.**

7.2. Diferenciar métodos de instancia y static

Java

```
public static int sumar(int a, int b) { return a + b; }
```

Esta declaración de la Unidad 6 describe un método estático que recibe sus operandos. En `Main` lo invocábamos sin crear un objeto de esa clase.

En cambio:

Java

```
public void mostrarInformacion() { ... }
```

es un método de instancia. Desde `main` lo invocamos sobre un objeto, por ejemplo `cuaderno.mostrarInformacion()`. Puede consultar el estado de **esa instancia**.

Pregunta decisiva: si ambos objetos comparten la definición del método, ¿por qué muestran información distinta? Porque sus **atributos de instancia tienen valores distintos**.

7.3. ¿A qué entidad corresponde cada responsabilidad?

Antes de convertir una acción en método, preguntemos qué información necesita y a quién corresponde principalmente.

Operación	Ubicación razonable para este problema
Consultar precio propio	`Producto`
Agregar o retirar su stock	`Producto`
Buscar entre cien productos	Inventario o gestor
Mostrar menú principal	`Main` o interfaz
Calcular recaudación de toda la tienda	Sistema de ventas

No es una regla universal. En un ejemplo pequeño, `Producto.mostrarInformacion()` ayuda a aprender métodos de instancia. En un sistema mayor quizá separaríamos presentación y modelo. **Lo que importa es justificar el reparto de responsabilidades.**

7.4. Pregunta para discutir sin Java

¿Tendría sentido agregar a `Producto` un método `registrarTodosLosEmpleados()`? ¿Qué entidad tendría que conocer para cumplir esa tarea? Propón un lugar más razonable o explica por qué la tarea queda fuera del sistema actual.

---

8. Descubrir el problema de los estados inválidos

La clase inicial permite modificar atributos directamente desde `Main`:

Java

```
cuaderno.precio = -500;
cuaderno.stock = -20;
```

El programa puede compilar, pero esos valores violan las reglas que decidimos para una tienda. El tipo `int` admite números negativos; no sabe por sí mismo que el stock debería ser no negativo.

8.1. Una regla de negocio

Para nuestro ejemplo adoptamos:

- Un producto tiene código positivo, nombre no vacío, precio no negativo y stock no negativo.
- Solo se agregan unidades en cantidades positivas.
- Solo se retiran cantidades positivas que no superen el stock disponible.
- Una operación rechazada **no modifica** los datos del producto.

Estas reglas provienen de los requisitos del problema, no de la sintaxis de Java.

8.2. Primera reacción: validar desde Main

Fragmento para analizar:

Fragmento Java

```
if (nuevoPrecio >= 0) {
    cuaderno.precio = nuevoPrecio;
}
```

Esta comprobación ayuda, pero ¿qué sucede si otro método modifica el producto y olvida validarlo? La regla queda dispersa. Queremos que **la propia clase controle las operaciones que afectan su estado**.

8.3. Una diferencia que nunca debemos perder

Control de acceso: decide desde dónde podemos leer o modificar directamente ciertos miembros.
Validación: comprueba si un valor o una operación cumple reglas del problema.
Encapsulamiento: organiza datos y operaciones, estableciendo una forma coherente de interactuar con el objeto.

Escribir `private` **no** hace que un número negativo deje de ser negativo. Primero limitaremos la modificación directa; después aprenderemos a validar cambios y creación.

8.4. El error intencional

Observa este constructor, con atributos privados:

Java

```
public Producto(int codigo, String nombre,
                double precio, int stock) {
    this.codigo = codigo;
    this.nombre = nombre;
    this.precio = precio;
    this.stock = stock;
}
```

¿Impedirá crear `new Producto(101, "Cuaderno", -500, -20)`? **No.** Aunque los atributos sean privados, el constructor mostrado no valida. Esta comprobación será parte esencial de nuestra discusión.

Salida de clase en papel: define con tus palabras por qué «atributo privado» y «atributo válido» no significan lo mismo.

---

9. Encapsulamiento básico: proteger y operar sobre el estado

Empezaremos por la versión **esencial**, usando datos iniciales válidos y métodos que no acepten operaciones inválidas. Su constructor todavía **no garantiza por sí solo** que todos los argumentos iniciales sean correctos; la siguiente sección resolverá esa limitación.

9.1. Clase Producto: versión esencial

Producto.java completo — versión A:

Java

```
public class Producto {
    private int codigo;
    private String nombre;
    private double precio;
    private int stock;

    // En esta primera versión usamos datos iniciales válidos.
    public Producto(int codigo, String nombre,
                    double precio, int stock) {
        this.codigo = codigo;
        this.nombre = nombre;
        this.precio = precio;
        this.stock = stock;
    }

    public int getCodigo() {
        return codigo;
    }
```

```
    public String getNombre() {
        return nombre;
    }

    public double getPrecio() {
        return precio;
    }

    public int getStock() {
        return stock;
    }

    public boolean cambiarPrecio(double nuevoPrecio) {
        boolean realizado = false;
        if (nuevoPrecio >= 0) {
            precio = nuevoPrecio;
            realizado = true;
        }
        return realizado;
    }

    public boolean agregarStock(int cantidad) {
        boolean realizado = false;
        if (cantidad > 0) {
            stock = stock + cantidad;
            realizado = true;
        }
        return realizado;
    }

    public boolean retirarStock(int cantidad) {
        boolean realizado = false;
        if (cantidad > 0 && cantidad <= stock) {
            stock = stock - cantidad;
            realizado = true;
        }
        return realizado;
    }

    public void mostrarInformacion() {
        System.out.println("Código: " + codigo);
        System.out.println("Nombre: " + nombre);
        System.out.println("Precio: " + precio);
        System.out.println("Stock: " + stock);
    }
}
```

Main.java completo para probarla:

Java

```
public class Main {
    public static void main(String[] args) {
        Producto cuaderno = new Producto(101, "Cuaderno", 120, 15);
```

```
        boolean pudoAgregar = cuaderno.agregarStock(5);
        boolean pudoRetirar = cuaderno.retirarStock(3);
        boolean retiroInvalido = cuaderno.retirarStock(30);

        System.out.println("Agregado: " + pudoAgregar);
        System.out.println("Retirado: " + pudoRetirar);
        System.out.println("Retiro de 30: " + retiroInvalido);
        System.out.println("Stock final: " + cuaderno.getStock());
    }
}
```

Salida:

Texto

```
Agregado: true
Retirado: true
Retiro de 30: false
Stock final: 17
```

El stock empieza en 15, aumenta a 20 y baja a 17. El retiro de 30 se rechaza: el objeto conserva 17.

9.2. ¿Por qué boolean?

`retirarStock` necesita comunicar si realizó la operación. Devolver `true` o `false` permite que `Main` decida qué mensaje mostrar, sin que `Producto` necesite controlar toda la interacción con el usuario.

Por ejemplo:

Java

```
boolean retirado = cuaderno.retirarStock(4);
if (retirado) {
    System.out.println("Operación realizada");
} else {
    System.out.println("Operación rechazada");
}
```

Fragmento para analizar: requiere que exista un objeto `cuaderno`.

9.3. Límites conscientes de la versión A

Es una implementación didáctica, no un sistema comercial robusto. La versión A:

- **supone** que los valores enviados al constructor ya son válidos;
- no contempla todos los valores especiales posibles de `double`;
- no protege todavía contra desbordamiento si el stock se acerca al máximo de `int`;
- no impide que otro producto tenga el mismo código.

No la presentaremos como garantía completa de invariantes. Su objetivo es aprender **atributos privados + métodos de instancia que comprueban una regla sencilla antes de modificar datos**.

9.4. Predecir antes de ejecutar

Stock inicial 12. ¿Cuál será el estado tras `agregarStock(5)`, `retirarStock(8)` y `retirarStock(20)`? Escribe el valor y el booleano devuelto en cada paso.

Operación	Resultado esperado	Stock después
Inicio	—	12
`agregarStock(5)`	`true`	17
`retirarStock(8)`	`true`	9
`retirarStock(20)`	`false`	9

---

10. Validar la creación, decidir getters y setters

Nuestra versión A controla ciertas modificaciones posteriores, pero el constructor todavía puede recibir datos incorrectos. **El aprendizaje siguiente no consiste en agregar métodos por costumbre, sino en decidir cómo proteger las reglas desde el comienzo.**

10.1. Tres niveles deliberadamente distintos

A — Esencial: clase con atributos privados, constructor usando argumentos válidos, getters necesarios y operaciones de stock validadas. Conocer y explicar sus límites.

B — Consolidación: además, el constructor verifica datos iniciales. Si no cumplen las reglas, rechaza la creación. La clase deja de depender solamente de que `Main` recuerde validar primero.

C — Robustez adicional: tratar valores excepcionales de `double`, desbordamiento de `int` y otros casos técnicos. No será requisito mínimo para comprender POO.

10.2. Cómo rechazar una creación inválida

Para una versión B con validación interna podemos utilizar la excepción estándar `IllegalArgumentException`. No desarrollaremos un sistema de captura de excepciones ni excepciones personalizadas; solo reconoceremos que un constructor puede **rechazar argumentos inválidos**.

Producto.java completo — versión B:

Java

```
public class Producto {
    private int codigo;
    private String nombre;
    private double precio;
    private int stock;

    public Producto(int codigo, String nombre,
                    double precio, int stock) {
        if (codigo <= 0 || nombre == null ||
            nombre.trim().isEmpty() || precio < 0 || stock < 0) {
            throw new IllegalArgumentException("Datos iniciales inválidos");
        }

        this.codigo = codigo;
        this.nombre = nombre;
        this.precio = precio;
        this.stock = stock;
    }

    public int getCodigo() {
        return codigo;
    }

    public String getNombre() {
        return nombre;
    }

    public double getPrecio() {
        return precio;
    }

    public int getStock() {
        return stock;
    }

    public boolean cambiarPrecio(double nuevoPrecio) {
        boolean realizado = false;
        if (nuevoPrecio >= 0) {
            precio = nuevoPrecio;
            realizado = true;
        }
        return realizado;
    }

    public boolean agregarStock(int cantidad) {
        boolean realizado = false;
        if (cantidad > 0) {
            stock += cantidad;
            realizado = true;
        }
        return realizado;
    }

    public boolean retirarStock(int cantidad) {
        boolean realizado = false;
        if (cantidad > 0 && cantidad <= stock) {
            stock -= cantidad;
```

```
            realizado = true;
        }
        return realizado;
    }

    public void mostrarInformacion() {
        System.out.println("Código: " + codigo);
        System.out.println("Nombre: " + nombre);
        System.out.println("Precio: " + precio);
        System.out.println("Stock: " + stock);
    }
}
```

Esta versión mejora el control de creación **para los valores ordinarios previstos en el ejercicio**. No resuelve aún todos los posibles problemas numéricos: por ejemplo, `NaN` puede escapar a la comparación `precio < 0`, y la suma de enteros puede desbordarse. Por eso reservamos el término **robustez completa** para una ampliación; no prometemos una garantía que la implementación no cumple.

Main.java para una prueba de creación válida:

Java

```
public class Main {
    public static void main(String[] args) {
        Producto lapiz = new Producto(102, "Lápiz", 35, 40);
        lapiz.mostrarInformacion();
    }
}
```

¿Qué sucede con datos inválidos?

Error intencional para analizar, no para dejar en la versión final:

Java

```
Producto invalido = new Producto(103, "Mochila", -10, 8);
```

El constructor lanza `IllegalArgumentException` y **no completa la creación** de un objeto con esos argumentos. En esta unidad no será necesario capturar esa excepción; el objetivo es reconocer que el constructor puede proteger las condiciones de creación.

10.3. Getters: consultar no es modificar

`getStock()` devuelve la cantidad actual, pero no ofrece acceso para asignar directamente al atributo privado. Un getter tiene sentido cuando otra parte del programa **necesita consultar** ese dato.

10.4. Setters: no generarlos automáticamente

¿Debemos ofrecer `setCodigo`? Si el código identifica al producto y no debe cambiar durante su vida, **no**. ¿Debemos ofrecer un `setStock` que reemplace cualquier cantidad? Para nuestra tienda resultan más expresivas las operaciones `agregarStock` y `retirarStock`, que validan cantidades concretas.

Atributo	Consulta posible	Cambio permitido en este modelo
`codigo`	`getCodigo()`	No se modifica luego de crear
`nombre`	`getNombre()`	No es requisito de esta versión
`precio`	`getPrecio()`	`cambiarPrecio(...)` con validación
`stock`	`getStock()`	`agregarStock(...)` y `retirarStock(...)`

Un getter o setter no es obligatorio por el solo hecho de existir un atributo. El acceso debe surgir de las funciones reales del programa.

10.5. Versión C: robustez técnica (lectura opcional)

En sistemas reales hay detalles adicionales. `double` puede almacenar `NaN` o infinito; para dinero real también tiene límites de precisión y normalmente se elegiría una representación decimal apropiada. `int` puede desbordarse si sumamos cantidades demasiado grandes. Podemos reforzar operaciones así:

Java

```
if (nuevoPrecio >= 0 && Double.isFinite(nuevoPrecio)) {
    precio = nuevoPrecio;
}
```

Java

```
if (cantidad > 0 && stock <= Integer.MAX_VALUE - cantidad) {
    stock += cantidad;
}
```

Estos son **fragmentos para analizar**, no un nuevo programa completo. Si los utilizamos, también habría que aplicar coherentemente las verificaciones en el constructor y en los demás métodos. No los exigiremos en la versión esencial.

La unicidad global de códigos tampoco se logra validando un objeto aislado: comprobar que no exista otro producto con el mismo código requiere conocer **el conjunto de productos**, responsabilidad razonable del inventario o gestor.

10.6. Simulador: operaciones y estados válidos

> Un Producto con stock 15 acepta agregarStock(5) y queda en 20; desde el estado inicial, retirarStock(20) se rechaza y conserva 15. private limita el acceso; las operaciones y el constructor necesitan validar sus reglas.
>
> Predice el resultado y contrástalo con los ejemplos del capítulo.

10.7. Mini-defensa

¿Por qué `private` no valida por sí solo un precio? ¿Por qué preferimos `retirarStock(3)` a permitir `producto.stock = 12` desde cualquier parte? ¿Qué condición debe comprobarse al construir un objeto y cuál al retirar unidades?

---

11. Varios objetos, arreglos y relaciones sencillas

Ya sabemos que las clases permiten crear muchos objetos. Los arreglos de la Unidad 5 también pueden conservar **referencias a objetos**.

11.1. Reservar posiciones no crea los productos

Java

```
Producto[] productos = new Producto[3];
```

Esta instrucción crea un arreglo con tres posiciones, pero todavía **no crea objetos** `Producto` en ellas. Sus valores iniciales son `null`.

Momento	`productos[0]`	`productos[1]`	`productos[2]`
Crear arreglo	`null`	`null`	`null`
Crear producto 101	Referencia a 101	`null`	`null`
Crear producto 102	Referencia a 101	Referencia a 102	`null`
Crear producto 103	Referencia a 101	Referencia a 102	Referencia a 103

Intentar ejecutar `productos[1].mostrarInformacion()` cuando todavía es `null` produce una `NullPointerException`. No necesitamos estudiar ahora cómo capturarla, pero sí entender **por qué sucede**.

11.2. Programa completo de recorrido

Utiliza la clase `Producto` de la sección 10 (versión A o B) y este `Main.java`:

Java

```
public class Main {
    public static void main(String[] args) {
        Producto[] productos = new Producto[3];

        productos[0] = new Producto(101, "Cuaderno", 120, 15);
        productos[1] = new Producto(102, "Lápiz", 35, 40);
        productos[2] = new Producto(103, "Mochila", 950, 8);

        for (int i = 0; i < productos.length; i++) {
            productos[i].mostrarInformacion();
        }
    }
}
```

El ciclo recorre **referencias** y, en cada posición ocupada, invoca el método del objeto correspondiente. Las herramientas anteriores siguen formando parte de la solución.

11.3. Relaciones sencillas sin sobrecargar la unidad

En una biblioteca, un `Prestamo` puede necesitar conocer qué `Libro` está asociado con él. Podemos representar conceptualmente:

Texto

```
LIBRO
  codigo
  titulo

PRESTAMO
  numero
  libro: referencia a Libro
```

Una relación entre objetos surge porque **el problema la necesita**. Por ahora basta con identificarla y explicarla; no exigiremos construir un sistema completo de préstamos con varias clases relacionadas.

11.4. ¿Quién busca un producto entre muchos?

La operación de revisar un conjunto de productos no tiene por qué pertenecer a cada producto individual. Un inventario podría conservar y recorrer varios objetos; en nuestro ejemplo introductorio también podemos hacer una búsqueda sencilla en `Main`.

Si el sistema exige **códigos únicos**, la comprobación debe realizarse en el contexto que conoce a todos los productos. Un solo objeto `Producto` no puede verificar por sí mismo que ninguno de los demás repita su

código.

Checkpoint: explica por qué `new Producto[3]` y tres expresiones `new Producto(...)` realizan tareas distintas.

---

12. Mini-sistema integrador: productos con responsabilidades

El proyecto final de esta unidad será pequeño, cerrable y de consola. La novedad que evaluaremos no es la cantidad de opciones del menú, sino **la calidad del modelo y la comprensión de su funcionamiento**.

12.1. Primero modelar

Situación: una tienda administra productos. Cada producto tiene código, nombre, precio y stock. Queremos consultarlo, sumar unidades y retirarlas cuando corresponda.

Antes de escribir Java responde:

- **Entidad:** ¿qué representa cada objeto?
- **Atributos:** ¿qué información necesita conservar?
- **Comportamientos:** ¿qué operaciones afectan su propio estado?
- **Reglas:** ¿qué cambios son válidos y cuáles deben rechazarse?
- **Otras responsabilidades:** ¿qué tareas corresponden al programa principal y no al producto?

12.2. Versión A: esencial

Producto: clase `Producto` con cuatro atributos, un constructor, un método `mostrarInformacion` y **dos objetos** creados desde `Main`. Debe poder demostrarse que modificar el stock de un objeto no cambia el otro.

Esta versión verifica abstracción, clase e instancia, constructor, estado independiente y método de instancia. Es la evidencia mínima de este bloque.

Pseudocódigo A

Texto

```
CLASE Producto
    ATRIBUTOS: codigo, nombre, precio, stock
    CONSTRUCTOR(datos): asignar datos a atributos
    METODO mostrarInformacion:
        mostrar los cuatro atributos
FIN CLASE

INICIO
```

```
    CREAR cuaderno con sus datos
    CREAR lapiz con sus datos
    MOSTRAR información de cuaderno
    MOSTRAR información de lapiz
FIN
```

Java completo A — Producto.java

Java

```
public class Producto {
    int codigo;
    String nombre;
    double precio;
    int stock;

    public Producto(int codigo, String nombre,
                    double precio, int stock) {
        this.codigo = codigo;
        this.nombre = nombre;
        this.precio = precio;
        this.stock = stock;
    }

    public void mostrarInformacion() {
        System.out.println(codigo + " - " + nombre);
        System.out.println("Precio: " + precio);
        System.out.println("Stock: " + stock);
    }
}
```

Java completo A — Main.java

Java

```
public class Main {
    public static void main(String[] args) {
        Producto cuaderno = new Producto(101, "Cuaderno", 120, 15);
        Producto lapiz = new Producto(102, "Lápiz", 35, 40);

        cuaderno.mostrarInformacion();
        lapiz.mostrarInformacion();

        // En esta versión inicial el acceso es de paquete.
        cuaderno.stock = 10;
        System.out.println("Cuaderno: " + cuaderno.stock);
        System.out.println("Lápiz: " + lapiz.stock);
    }
}
```

Las últimas líneas deben mostrar `10` para el cuaderno y `40` para el lápiz. Antes de avanzar, cada estudiante explica por qué.

12.3. Versión B: consolidación

Sustituye `Producto.java` por la versión de la sección 9 (o de la sección 10 para agregar validación inicial). Los atributos son privados y los cambios deben realizarse mediante operaciones controladas.

Main.java completo B:

Java

```
public class Main {
    public static void main(String[] args) {
        Producto cuaderno = new Producto(101, "Cuaderno", 120, 15);
        Producto lapiz = new Producto(102, "Lápiz", 35, 40);

        boolean agregado = cuaderno.agregarStock(5);
        boolean retirado = cuaderno.retirarStock(3);
        boolean imposible = lapiz.retirarStock(100);

        System.out.println("Agregado: " + agregado);
        System.out.println("Retirado: " + retirado);
        System.out.println("Retiro imposible: " + imposible);
        System.out.println("Stock cuaderno: " + cuaderno.getStock());
        System.out.println("Stock lápiz: " + lapiz.getStock());
    }
}
```

Salida esperada:

Salida esperada

```
Agregado: true
Retirado: true
Retiro imposible: false
Stock cuaderno: 17
Stock lápiz: 40
```

La operación rechazada no modifica el stock del lápiz. La independencia de los objetos se mantiene.

12.4. Versión C: ampliación

Agregar un arreglo de productos y un pequeño menú de consola con `do...while` y `Scanner`. Opciones posibles: mostrar productos, buscar por código, agregar stock, retirar stock y salir.

Antes de construir el menú, pregunta qué parte conoce el conjunto de productos y qué parte conoce el stock de cada producto. No conviertas la clase `Producto` en responsable de todo el sistema.

No se requieren archivos, interfaces gráficas, bases de datos, herencia ni colecciones dinámicas.

12.5. Casos de prueba de operaciones (reiniciar estado en cada caso)

Estado inicial	Operación	Resultado	Estado final
Stock 15	Agregar 5	`true`	20
Stock 15	Retirar 3	`true`	12
Stock 15	Retirar 20	`false`	15
Stock 15	Retirar 0	`false`	15
Stock 15	Retirar -2	`false`	15
Precio 120	Cambiar a 150	`true`	150
Precio 120	Cambiar a -20	`false`	120

Los casos son **independientes**. No ejecutes las operaciones una detrás de otra esperando que el estado se reinicie automáticamente.

12.6. Cambio de requisito y defensa

El comercio decide que **deben quedar al menos dos unidades** después de cada retiro. ¿Dónde corresponde implementar la nueva regla? ¿Qué condición cambiarías en `retirarStock`? ¿Qué casos probarías?

Como la nueva condición debe dejar dos unidades disponibles, en este modelo con stock no negativo necesitamos comprobar que la cantidad sea positiva y no supere `stock - 2`, pero cuidando que `stock` sea al menos 2:

Java

```
if (cantidad > 0 && stock >= 2 && cantidad <= stock - 2) {
    stock -= cantidad;
    realizado = true;
}
```

Fragmento para analizar: corresponde al cuerpo de `retirarStock`; no es un programa autónomo. El estudiante debe explicar por qué la regla se implementa allí y no solamente en un mensaje de `Main`.

---

13. Errores típicos: de sintaxis, de ejecución y de diseño

Debemos aprender a **leer, predecir y corregir** errores, no solamente a copiar soluciones.

13.1. Confundir definición de clase y objeto creado

Definir `public class Producto { ... }` no crea automáticamente tres productos. Para obtener cada instancia utilizamos una construcción adecuada como `new Producto(...)`.

13.2. Olvidar new

Fragmento incorrecto:

Java · ejemplo incorrecto

```
Producto producto;
producto.mostrarInformacion();
```

La variable local todavía no recibió una referencia. Java detecta que se intenta utilizar una variable local no inicializada. Antes de invocar un método hay que crear o asignar un objeto válido.

13.3. Confundir dos instancias con dos referencias

Java

```
Producto b = a;
```

no crea un segundo producto. Si queremos otro objeto, normalmente utilizamos otra invocación de `new` con los datos pertinentes.

13.4. Utilizar un constructor que no existe

Si el único constructor recibe cuatro argumentos, `new Producto()` deja de ser una forma válida de creación. Debemos proporcionar los cuatro argumentos o declarar otro constructor adecuado.

13.5. Poner void en el constructor

Incorrecto:

Java

```
public void Producto(int codigo) { ... }
```

Esto declara un método, no un constructor. Un constructor **no** declara tipo de retorno.

13.6. Confundir atributos y parámetros

En `this.stock = stock;`, a la izquierda tenemos el atributo del objeto y a la derecha el parámetro del constructor. Si escribimos solamente `stock = stock;` con ambos nombres iguales, no inicializaremos el atributo como pretendemos.

13.7. Acceder a atributos privados desde Main

Con `private int stock`, una instrucción externa como `producto.stock = -2` no compila. Debe utilizarse una operación pública permitida por el diseño.

13.8. Confundir privacidad y validación

Un constructor que asigna `stock = -2` puede crear un estado inválido **aunque el atributo sea privado**. Necesitamos verificar las reglas en los puntos de modificación pertinentes, incluida la creación.

13.9. Setters que anulan las reglas

Un `setStock(int valor)` que acepta cualquier número puede anular el sentido de `agregarStock` o `retirarStock`. Pregunta qué modificaciones necesita realmente el programa.

13.10. Ignorar el resultado de una operación

Si `retirarStock` devuelve `false`, el retiro fue rechazado y el stock no cambió. Un mensaje de «Operación realizada» sin comprobar ese resultado sería engañoso.

13.11. Confundir métodos static y de instancia

En nuestros ejemplos, `main` es estático. `mostrarInformacion` pertenece a un objeto concreto y se invoca, por ejemplo, mediante `cuaderno.mostrarInformacion()`.

13.12. Creer que cada variable significa un objeto distinto

Las referencias pueden apuntar al mismo objeto. Debes seguir las instrucciones `new` y asignaciones, no contar solamente nombres.

13.13. Reservar un arreglo y olvidar crear sus objetos

Java

```
Producto[] productos = new Producto[3];
```

crea posiciones con `null`; no crea automáticamente tres productos. Invocar un método a través de una posición aún nula produce un error de ejecución.

13.14. Crear una clase para cada sustantivo

En «el estudiante consulta un libro desde una computadora», la computadora podría quedar fuera del sistema y la consulta podría ser una operación. **Modelar exige seleccionar según la finalidad.**

13.15. Entregar demasiadas responsabilidades a una clase

Una clase `Libro` que calcula salarios, administra usuarios y controla todos los préstamos podría estar mezclando funciones ajenas a su entidad. Debemos justificar quién necesita esos datos y quién debe realizar cada operación.

13.16. Creer que un objeto controla información global que no conoce

Un producto no puede garantizar por sí mismo que no exista otro producto con el mismo código si no conoce el conjunto de productos. Esa regla corresponde razonablemente al inventario o gestor.

13.17. Elegir atributos irrelevantes

Registrar el peso de cada tornillo de una bicicleta en un sistema de alquiler sencillo añade complejidad sin responder a un requisito. Una clase representa una **abstracción orientada a un propósito**, no la realidad completa.

13.18. Depurar el diseño y no solo el código

Si el programa compila pero resulta difícil de modificar, pregunta:

1. ¿Qué entidades necesita representar realmente?
2. ¿Qué atributos son relevantes para los requisitos?
3. ¿A quién corresponde cada operación?
4. ¿Qué reglas deben proteger el estado?
5. ¿Dos variables refieren a una misma instancia?
6. ¿La clase está asumiendo responsabilidades de todo el sistema?
7. ¿Qué cambio pequeño permitiría mejorar el modelo?

Actividad breve: explica por qué la clase `Libro` con `calcularSueldos()` y `registrarTodosLosUsuarios()` puede ser un error de diseño aunque esos métodos compilen.

13.19. Laboratorio guiado de modelado sin clase predeterminada

Situación nueva. Una institución quiere registrar el préstamo de equipos portátiles. Cada equipo tiene un identificador y puede estar disponible o prestado. Se necesita registrar el préstamo y la devolución. No se pide todavía un sistema de usuarios, multas o archivos.

Primera etapa — decisión, sin Java. Propón qué entidad merece una clase y qué atributos necesita. Describe dos comportamientos y dos estados válidos. Explica qué aspectos deliberadamente dejas fuera. No conviertas automáticamente cada sustantivo en clase.

Pista 1. Pregunta qué información necesita conservar el programa *entre operaciones*. **Pista 2.** Un objeto `Equipo` podría conservar `codigo` y `prestado`, y ofrecer `prestar()` y `devolver()`; decide si esas operaciones deben informar éxito o fracaso.

Segunda etapa — modelo contrastable. Es razonable proponer `Equipo(codigo, prestado)`; estado inicial disponible; `prestar()` solo cambia a prestado si estaba disponible; `devolver()` solo cambia a disponible si estaba prestado. Este es un ejemplo posible, no la única solución válida. Si tu modelo incorpora otras entidades, justifica qué requisito las hace necesarias.

Tercera etapa — Java. Crea `Equipo.java` y `Main.java`. Construye dos objetos y demuestra que prestar uno no modifica el otro. Introduce una operación inválida y comprueba que el objeto conserva su estado. La evidencia no consiste solamente en compilar: explica qué condición protegió el estado.

Cambio de requisito, primero en papel: ahora cada equipo registra el número de veces que fue prestado. ¿Es un atributo o una clase? ¿Qué operación debe actualizarlo y cuándo? Ajusta el modelo antes de modificar código; comprueba que un intento fallido de préstamo no incremente el contador.

Error de diseño para corregir: una clase `Equipo` contiene `contrasenaDirector`, `calcularSalarios()` y `mostrarMenuGeneral()`. Determina qué responsabilidades no corresponden principalmente al equipo y por qué, aunque Java permitiera escribir esos métodos allí.

Defensa individual (90 segundos): muestra tu modelo, justifica un atributo descartado, explica el estado independiente de dos objetos y predice el resultado de un préstamo rechazado. Si lo hiciste en pareja, intercambia conductor y navegante cada 10–15 minutos; cada integrante explica una parte y presenta una prueba.

13.20. Pistas y soluciones en capas para las actividades A–W

Cuando una actividad pida «diseña una clase», la página debe ofrecer tres niveles, inicialmente ocultos: **pista de entidad** («¿qué necesita representar el sistema?»), **pista de responsabilidad** («¿qué cambios pertenecen al objeto y cuáles a la aplicación?») y **modelo contrastable** (atributos, operaciones, reglas y una justificación). La solución Java completa se muestra solamente después de realizar un intento. En ejercicios de varios objetos, la solución debe incluir siempre un trazado de estados y al menos una prueba donde una operación sea rechazada.

No calificaremos mejor una solución solo por tener más clases. Un modelo sencillo que cubre los requisitos y puede explicarse vale más que una estructura compleja copiada sin comprenderla.

---

14. Actividades A–W: aprender con productos pequeños

Las **23 actividades** se mantienen como banco de prácticas. No son una lista que todos deban completar de corrido. Las organizamos en cuatro mini-ciclos de dos a cuatro clases, cada uno con evidencia cerrable.

Mini-ciclo 1. Observar, seleccionar y justificar (A–D)

Actividad A. Encontrar entidades

Una biblioteca escolar registra libros, lectores y préstamos. Cada libro tiene código y título; cada lector tiene identificación y nombre; el préstamo relaciona un libro con un lector. Propón entidades relevantes, dos atributos de cada una y una responsabilidad. Justifica si «disponibilidad» necesita ser una clase o podría ser un dato o resultado.

Actividad B. Dos modelos de la misma realidad

Diseña en papel los datos relevantes de una bicicleta para un sistema **de alquiler** y otro **de mantenimiento**. Elige al menos tres atributos por caso y excluye dos datos que no sean relevantes. Explica por qué los modelos difieren. **No se escribe Java**.

Actividad C. Clase frente a objetos

Para la clase `Estudiante`, consideramos a Ana (101), Luis (102) y Sofía (103). Señala clase, instancias, atributos comunes en estructura y valores particulares. ¿Qué cambia si dos variables guardan referencias al mismo estudiante?

Actividad D. Distinguir datos, operaciones y otras responsabilidades

En una tienda aparecen `codigo`, `precio`, `nombre`, `stock`, «agregar unidades», «retirar unidades», «calcular la recaudación total» y «mostrar menú». Clasifica cada elemento como dato o acción y propone a quién corresponde cada operación. No asignes todo automáticamente a `Producto`.

Evidencia del mini-ciclo: una ficha de modelado con entidad, propósito, atributos, comportamientos y una decisión justificada de exclusión.

Mini-ciclo 2. De la clase al objeto (E–J)

Actividad E. Primera clase

Construye `Libro` con código entero, título, autor y cantidad de páginas. En esta primera versión utiliza atributos de acceso de paquete para practicar la estructura. Escribe `Libro.java`.

Actividad F. Crear dos libros

En `Main.java`, crea dos objetos diferentes con `new`. Asigna valores y muestra sus títulos. Comprueba qué sucede si modificas solo uno de ellos.

Actividad G. Introducir un constructor

Reescribe `Libro` para que los cuatro datos se reciban al crear el objeto. Antes de programar, explica qué problema resuelve respecto de las asignaciones separadas.

Actividad H. Agregar un comportamiento

Define `mostrarInformacion()` como método de instancia de `Libro`. Invócalo sobre ambos objetos. Predice qué datos mostrará cada invocación.

Actividad I. Predecir independencia

Un programa crea dos productos con stocks 10 y 20; después modifica solamente el primero. Realiza un trazado con ambos objetos y explica por qué el segundo conserva su valor.

Actividad J. Detectar referencias compartidas

Analiza con la clase `Producto` de las secciones 9 o 10:

Java

```
Producto a = new Producto(101, "Cuaderno", 120, 15);
Producto b = a;
b.agregarStock(5);
System.out.println(a.getStock());
```

Predice la salida. ¿Cuántos objetos se crearon? ¿Cambiaría la respuesta si `b` se hubiera creado con otra expresión `new`?

Evidencia del mini-ciclo: `Libro.java` y `Main.java`, con dos objetos y explicación individual de un estado.

Mini-ciclo 3. Proteger el estado y razonar operaciones (K–O)

Actividad K. Detectar una regla violada

En una clase inicial, el código externo ejecuta `producto.stock = -50`. Explica por qué es inválido según el problema. Distingue el papel del modificador de acceso y el de la validación.

Actividad L. Agregar stock

Implementa `agregarStock(int cantidad)` con retorno booleano. Acepta solamente cantidades positivas. Prueba 5, 0 y -3. Registra resultado y estado anterior/posterior.

Actividad M. Retirar stock

Implementa `retirarStock(int cantidad)` que rechace cero, negativos y retiros superiores al disponible. Parte de un stock 10 y prueba 3, 10, 11 y -1 **reiniciando el estado para cada caso**.

Actividad N. Decidir qué consultar y qué modificar

Para `Producto`, elige qué getters necesita un programa que consulta código, precio y stock. Justifica por qué no necesariamente corresponde ofrecer `setCodigo` ni `setStock`. Propón una operación con nombre significativo para modificar stock.

Actividad O. Predicción de operaciones consecutivas

Un producto comienza con stock 12. Se ejecutan `agregarStock(5)`, `retirarStock(8)` y `retirarStock(20)`. Completa una tabla con booleano devuelto y stock después de cada paso. Explica por qué el último rechazo no altera el objeto.

Evidencia del mini-ciclo: clase con atributos privados y operaciones controladas más tabla de pruebas válidas e inválidas.

Mini-ciclo 4. Transferir el modelo a un problema nuevo (P–W)

Actividad P. Modelar una cuenta sin Java

Una cuenta tiene número, titular y saldo. Puede consultar saldo, depositar importes positivos y retirar si existe saldo suficiente. Identifica entidad, atributos, comportamientos y reglas. ¿Qué datos deben protegerse? ¿Quién verificaría la unicidad de números de cuenta si hubiera muchas?

Actividad Q. Construir Cuenta

Implementa `Cuenta` con atributos privados, constructor, consulta de saldo, depósito y retiro. Rechaza cero, negativos y retiros que excedan el saldo. Prueba las operaciones. Para este ejercicio utiliza importes sencillos con `double`; no se exige resolver todavía problemas de precisión monetaria.

Actividad R. Modelo de vehículo

Una carrera registra vehículos con identificador, nombre y velocidad actual. Pueden acelerar, frenar y consultar velocidad. Propón reglas para impedir velocidad negativa; implementa una clase simple. No utilices herencia.

Actividad S. Arreglo de productos

Crea tres objetos `Producto`, guárdalos en un arreglo y recórrelo. Cambia solamente el stock del segundo. Predice el estado de los tres antes de ejecutar.

Actividad T. Mini-sistema de inventario

Desarrolla el integrador en una de sus versiones: A (crear y consultar), B (operaciones y validación) o C (arreglo, búsqueda y menú). Entrega modelo, código, pruebas y defensa.

Actividad U. Un problema sin clases indicadas

Un torneo necesita registrar equipos y jugadores. Cada equipo tiene nombre e identificador; cada jugador tiene nombre y número de camiseta, y pertenece a un equipo. Debe poder consultarse quiénes integran

cada equipo. **No escribas Java**. Propón entidades, atributos, comportamientos y relaciones. Señala al menos una decisión discutible y justifica tu elección.

Actividad V. Transformar variables independientes

Recibe un programa con `codigo1`, `nombre1`, `precio1`, `stock1`, `codigo2`, etc. Construye un modelo `Producto`, crea dos objetos, traslada un comportamiento pertinente y verifica que las salidas principales se mantienen. Explica qué mejoró y qué sigue siendo discutible.

Actividad W. Defensa de un diseño

Prepara una explicación de 90 segundos de una clase que hayas creado: entidad, finalidad, atributos incluidos y descartados, comportamientos, reglas, estado inicial y prueba. Responde un cambio de requisito propuesto por el docente.

Evidencia del mini-ciclo: mini-sistema pequeño y defensa individual obligatoria. Las actividades U y W comprueban que el estudiante sabe **modelar**, no solamente reproducir código.

---

15. Evaluación de la unidad y cierre de Programación I

15.1. Aprendizajes esenciales

Para considerar alcanzada la base de POO, cada estudiante debería poder:

1. Identificar una entidad relevante según el problema y justificar qué información conservar.
2. Distinguir clase, objeto, atributo y comportamiento.
3. Crear una clase sencilla y dos instancias distintas.
4. Explicar cómo se ejecuta un constructor y qué datos recibe.
5. Invocar un método de instancia y relacionarlo con el estado correcto.
6. Predecir si dos variables apuntan a objetos distintos o al mismo objeto.
7. Explicar por qué conviene proteger algunos atributos.
8. Implementar al menos una operación que valide antes de modificar el estado.
9. Probar una operación aceptada y una rechazada.
10. Defender una decisión de abstracción y adaptarse a un cambio pequeño.

15.2. Consolidación y ampliación

Como profundización: constructor que valida datos iniciales, getters seleccionados por propósito, arreglos de objetos, búsqueda por código y relaciones sencillas entre clases. Como ampliación adicional: casos especiales numéricos y excepciones. No son requisitos mínimos: **herencia, polimorfismo, colecciones dinámicas, interfaces gráficas ni archivos**.

15.3. Rúbrica corta de observación

Criterio	Logrado	En proceso	Inicial
Abstracción	Selecciona y justifica entidades y atributos relevantes	Necesita preguntas para justificar	Copia clases sin relacionarlas con el problema
Clase/objeto	Distingue definición, instancias y estados	Confunde alguna referencia o estado	No diferencia clase de objeto
Construcción Java	Crea objetos, constructor y método coherentes	Requiere correcciones guiadas	No logra una estructura funcional
Responsabilidades y reglas	Sitúa una operación razonablemente y valida su cambio	Reglas o decisiones incompletas	Permite cambios inválidos sin detectarlos
Pruebas y defensa	Predice, prueba y explica decisiones	Explicación o pruebas parciales	No justifica ni comprueba el comportamiento

15.4. Producto final: pequeño, grupal y cerrable

El integrador podrá ser de productos, cuenta, libros o vehículos. Se desarrollará en aproximadamente **dos o tres clases** según el grupo. Cada pareja o equipo entrega **una solución**, con pseudocódigo/modelo, código y pruebas.

El programa debe tener al menos una clase propia, atributos, constructor, creación de objetos, un método de instancia y una regla de validación. La versión mínima **no** necesita menú ni varias clases. La versión ampliada puede incorporar un arreglo de objetos.

15.5. Defensa individual obligatoria

Cada estudiante deberá:

- Explicar qué entidad representó y por qué.
- Justificar dos atributos, uno incluido y uno descartado.
- Explicar un método y su condición de validación.
- Predecir un cambio de estado sobre un objeto concreto.
- Realizar un cambio pequeño en vivo o en papel, como modificar un criterio de retiro.

Para verificar comprensión y evitar entregas copiadas pueden utilizarse: explicación de 90 segundos, predicción de salida, completar huecos, corregir un bug, cambiar un requisito o trazado de objetos en papel.

15.6. Tres preguntas de salida

1. Clase frente a objeto. En `Producto a = new Producto(...); Producto b = new Producto(...);`, ¿cuántas instancias se crean? ¿Qué cambia si escribimos `Producto c = a`?

2. Regla de estado. Si un producto tiene stock 8, ¿qué debe suceder al solicitar `retirarStock(12)`? Explica qué dato debe conservarse.

3. Abstracción. Para un sistema de biblioteca escolar que solo busca libros por título, ¿necesitamos representar el salario del bibliotecario en la clase `Libro`? Justifica según la finalidad.

15.7. Pair programming y comprobación individual

Roles: **Conductor** (teclea) y **Navegante** (predice, lee, comprueba), con intercambio cada 10–15 minutos. Una entrega por pareja, un minuto de explicación por estudiante. Checklist rápido: turnos efectivos, comunicación, pruebas, intercambio de roles. Si se observa asimetría, aplicar un checkpoint individual: predecir estado, completar un método o explicar una decisión de modelado.

15.8. Autoevaluación

Marca **Sí / Todavía necesito ayuda**:

- Puedo explicar la diferencia entre una clase y dos objetos de esa clase.
- Puedo justificar por qué un atributo debe quedar protegido y cómo se modifica válidamente.

Reflexión breve: «La decisión de diseño que más tuve que pensar fue… porque…».

15.9. Cierre del recorrido anual

En Programación I aprendimos a analizar problemas, diseñar algoritmos, utilizar variables, condiciones y ciclos, conservar datos en arreglos y organizar procedimientos mediante métodos. Ahora incorporamos la capacidad de **modelar entidades** y asignarles datos y responsabilidades.

Los objetos no reemplazan las herramientas anteriores: se construyen con ellas. Una buena solución comienza por comprender qué necesita el problema y termina con una prueba que demuestre que el programa hace lo esperado. La defensa individual debe mostrar que el estudiante **entiende y puede modificar** su programa.
